

2. Data Analytics I

Create a Linear Regression Model using Python/R to predict home prices using “Food_Delivery_Times” Dataset.

1. Print all predictor variables and dependent variables in the data set.
2. Understand the relationship between each predictor variable and dependent variable. Draw the plot.
3. Build simple linear regression model for predicting food delivery times based on various influencing factors such as distance, weather, traffic conditions, and time of day. Keep 80% samples for training and remaining for testing.
4. Show the accuracy on the test set.
5. Build multiple linear regression model. Compare MSE of simple linear and multiple linear regression.

Implement linear regression algorithm with gradient descent optimization.

- The program should be generic, should work for any dataset
- Print all the predictor variables and dependent variables in the given dataset
- Understand the relationship between each predictor variable and the dependent variable; draw the plot.
- Keep 80% of samples for training and rest for testing
- Print the regression parameters after training.
- Show the accuracy on the test set

```
# Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
```

```

from sklearn.preprocessing import StandardScaler

# Load the dataset
df = pd.read_csv('Food_Delivery_Times.csv') # Replace with actual file path

# Step 1: Print all predictor variables and dependent variables
predictors = ['distance', 'weather', 'traffic_conditions', 'time_of_day'] # Example predictors
dependent = 'delivery_time' # Target variable

print("Predictor Variables:")
print(predictors)
print("\nDependent Variable:")
print(dependent)

# Step 2: Understand the relationship between each predictor and dependent variable (Plot relationships)
# Plotting pairwise relationships (scatter plot matrix)
import seaborn as sns
sns.pairplot(df, vars=predictors + [dependent])
plt.show()

# Step 3: Simple Linear Regression Model (Using distance as the predictor)
X_simple = df[['distance']] # Simple linear regression with distance
y = df[dependent]

# Train-test split (80% training, 20% testing)
X_train_simple, X_test_simple, y_train, y_test = train_test_split(X_simple, y, test_size=0.2,
random_state=42)

# Simple Linear Regression Model
simple_model = LinearRegression()
simple_model.fit(X_train_simple, y_train)

```

```

# Predicting on the test set
y_pred_simple = simple_model.predict(X_test_simple)

# Evaluating performance using Mean Squared Error (MSE)
mse_simple = mean_squared_error(y_test, y_pred_simple)
print("\nMSE for Simple Linear Regression (using distance):", mse_simple)

# Step 4: Multiple Linear Regression Model
X_multiple = df[predictors] # Multiple predictors: distance, weather, traffic, and time_of_day

# Train-test split (80% training, 20% testing)
X_train_multiple, X_test_multiple, y_train, y_test = train_test_split(X_multiple, y, test_size=0.2,
random_state=42)

# Multiple Linear Regression Model
multiple_model = LinearRegression()
multiple_model.fit(X_train_multiple, y_train)

# Predicting on the test set
y_pred_multiple = multiple_model.predict(X_test_multiple)

# Evaluating performance using Mean Squared Error (MSE)
mse_multiple = mean_squared_error(y_test, y_pred_multiple)
print("\nMSE for Multiple Linear Regression:", mse_multiple)

# Step 5: Compare MSE of Simple and Multiple Linear Regression
print("\nComparison of MSE:")
print(f"Simple Linear Regression MSE: {mse_simple}")
print(f"Multiple Linear Regression MSE: {mse_multiple}")

```

Step 6: Gradient Descent Implementation for Linear Regression

Gradient Descent Implementation for Linear Regression

```
def gradient_descent(X, y, lr=0.01, epochs=1000):
```

```
    m, n = X.shape
```

```
    theta = np.zeros(n)
```

```
    cost_history = []
```

```
    # Adding a bias term (column of 1s) to the feature matrix X
```

```
    X_b = np.c_[np.ones((m, 1)), X] # Adding bias term (X_0 = 1)
```

```
    for _ in range(epochs):
```

```
        # Predictions
```

```
        predictions = X_b.dot(theta)
```

```
        # Compute the gradient
```

```
        gradients = 2/m * X_b.T.dot(predictions - y)
```

```
        # Update the parameters (theta)
```

```
        theta = theta - lr * gradients
```

```
        # Compute the cost and store it for later
```

```
        cost = (1/m) * np.sum((predictions - y)**2)
```

```
        cost_history.append(cost)
```

```
    return theta, cost_history
```

Prepare data for Gradient Descent (standardizing the features)

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(df[predictors])
```

```
y_scaled = df[dependent].values
```

```

# Train using Gradient Descent

theta, cost_history = gradient_descent(X_scaled, y_scaled, lr=0.01, epochs=1000)

# Print the learned parameters (theta values)
print("\nGradient Descent Parameters (Theta):")
print(theta)

# Plot the cost history over iterations
plt.plot(cost_history)
plt.title("Cost History using Gradient Descent")
plt.xlabel("Epochs")
plt.ylabel("Cost")
plt.show()

# Predictions using Gradient Descent (on test set)
X_test_scaled = scaler.transform(X_test_multiple)
X_test_b = np.c_[np.ones((X_test_scaled.shape[0], 1)), X_test_scaled] # Add bias term
y_pred_gd = X_test_b.dot(theta)

# Evaluate Gradient Descent model (MSE)
mse_gd = mean_squared_error(y_test, y_pred_gd)
print("\nMSE for Gradient Descent Linear Regression:", mse_gd)

# Step 7: Show the accuracy on the test set (for Multiple Linear Regression)
print("\nAccuracy (R^2 score) for Multiple Linear Regression:",
multiple_model.score(X_test_multiple, y_test))

```